

UNIDAD 2:

SETUP

ENTORNO

CURSO

IA

```
01 import openai
02 from langchain import
03     ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10
11     respuesta = modelo.invoke(
12         prompt
13     )
14     return respuesta.content
15
16
17 if __name__ == "__main__":
18     prompt = "Explicame qué
19     es la inteligencia artificial"
20     print(generar_respuesta(prompt))
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep.Argentina 786, Sunchales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383



ENTORNO DE TRABAJO

Lenguaje de Programación:	Python
Librería de desarrollo IA:	TensorFlow
Entorno de Desarrollo (IDE):	VSCode
Herramienta Didáctica:	AIR Battle
Herramienta Didáctica:	Lienzo MNIST





Pipeline de Instalación del Entorno de Trabajo

Fase 1: Descarga de AIRBATTLE desde GitHub

El proceso inicia descargando la herramienta didáctica **AIRBATTLE** desde la sección de *Releases* del repositorio oficial:

<https://github.com/BrainStormAcademy/00-BRAINSTORM-CursolA>.

Al compilar o descargar el proyecto, notarás que existen 3 formatos de distribución del ejecutable, cada uno diseñado para un escenario diferente:

- **Self-Contained (Autocontenido):** Incluye la aplicación y todas las librerías necesarias del sistema operativo en un solo paquete. Se puede ejecutar en cualquier máquina sin necesidad de instalar entornos o componentes de desarrollo adicionales.
- **Framework Dependent (Dependiente del Entorno):** Es un archivo mucho más ligero porque no incluye las librerías base. Requiere que la máquina de destino ya tenga instalado el entorno de ejecución global correspondiente para poder arrancar.
- **Ready to Run (Listo para Ejecutar):** Una versión optimizada para mejorar el tiempo de inicio de la aplicación. Viene precompilada a código nativo de la máquina, lo que reduce la carga del procesador al abrir el programa por primera vez.





Pipeline de Instalación del Entorno de Trabajo

Fase 2: Automatización del Entorno con INSTALL_PYTHON_TF.bat

Al ejecutar **AIRBATTLE**, la herramienta creará automáticamente la estructura del laboratorio y un script crítico en la ruta:

C:\000-LABORATORIO_IA\1-SCRIPTS\0-INSTALACION_ENTORNO\INSTALL_PYTHON_TF.bat

¿Qué se logra al ejecutar este archivo .bat?

Este script automatiza por completo la preparación del sistema operativo para que sea compatible con proyectos de Inteligencia Artificial. Al ejecutarlo (preferentemente como **Administrador**), logra lo siguiente de forma transparente:

- **Validación del Sistema:** Escanea la computadora para detectar si ya existen versiones previas de Python instaladas, evitando conflictos y permitiendo la coexistencia segura de entornos.
- **Instalación de la Versión Perfecta (Python 3.11.9):** Descarga de forma segura e instala de manera silenciosa la versión exacta de Python que garantiza la máxima estabilidad y compatibilidad con **TensorFlow 2.16+** en Windows de 64 bits.
- **Configuración del Sistema (PATH Global):** Inyecta automáticamente los ejecutables en las variables de entorno de Windows. Esto permite invocar los comandos python y pip desde cualquier consola de comandos sin configurar nada a mano.
- **Instalación y Actualización de Herramientas de IA:** Incorpora pip (el gestor de paquetes de Python) y lo fuerza a actualizarse a su última versión estable, dejándolo listo para descargar librerías pesadas de Deep Learning.
- **Limpieza de Residuos:** Tras finalizar con éxito, borra todos los instaladores temporales para no ocupar espacio innecesario en el disco.



Resultado final: El laboratorio queda perfectamente calibrado y con los cimientos sólidos para ejecutar el siguiente paso (python SETUP_DEPENDENCIES.py), el cual descargará TensorFlow y MNIST.



Pipeline de Instalación del Entorno de Trabajo

Fase 3: Configuración Integral del Ecosistema de IA: (SETUP_DEPENDENCIES.py)

Esta fase representa la **calibración fina y armado del software**. Mientras que el archivo .bat anterior dejó los cimientos sólidos (instalando Python), este script de Python toma el control para instalar el motor matemático de IA, las herramientas de desarrollo y los sistemas de control de código.

El script automatiza el flujo de trabajo dividiéndose en **5 etapas clave**:

- **1. Validación de Intérprete y Gestor (PASO 1/5)**

- **Acción:** Detecta que la versión activa de Python sea compatible con entornos de Machine Learning (mínimo versión 3.9).
- **Control:** Asegura la presencia de pip (el gestor de paquetes) y fuerza su actualización inmediata para evitar errores de red al bajar librerías pesadas.

- **2. Ingesta de Ecosistema Científico y de IA (PASO 2/5)**

El script escanea e instala de forma silenciosa un abanico de librerías esenciales agrupadas por función:

- **Core de Inteligencia Artificial: TensorFlow** (diseño de redes neuronales), **NumPy** y **Pandas** (manipulación de datos y matrices numéricas de alta velocidad).
- **Visualización y Diagnóstico: Matplotlib** y **Seaborn** (gráficas para analizar el comportamiento del error en el entrenamiento).
- **Utilidades del Proyecto:** Librerías de comunicación como websockets (para conectar la IA con el entorno didáctico del juego) y herramientas de entorno como jupyter.





Pipeline de Instalación del Entorno de Trabajo

Fase 3: Configuración Integral del Ecosistema de IA: (SETUP_DEPENDENCIES.py)

• 3. Control de Versiones Automatizado (PASO 3/5)

- **Acción:** Verifica la existencia de **Git** en la máquina.
- **Solución inteligente:** Si el sistema no cuenta con él, utiliza los administradores de paquetes de Windows (winget o Chocolatey) para descargarlo e instalarlo de manera invisible.

Auditoría: Revisa si el desarrollador tiene configuradas sus credenciales globales (user.name y user.email).

• 4. Enlace Seguro con GitHub mediante OAuth (PASO 4/5)

- **Acción:** Instala **GitHub CLI (gh)**, la herramienta por consola oficial de GitHub.
- **Autenticación:** Ofrece abrir el navegador web en tiempo real para autorizar la cuenta del desarrollador de forma segura (OAuth).

Resultado: Configura el asistente de credenciales, permitiendo crear, clonar y subir repositorios a la nube desde la terminal sin ingresar contraseñas manualmente cada vez.

• 5. Preparación del Entorno de Desarrollo y Código (PASO 5/5)

- **IDE Automatizado:** Instala o valida **Visual Studio Code**.
- **Extensiones Especializadas(DESABILITADO):** Prepara el editor inyectándole herramientas críticas de desarrollo:
Soporte oficial de Python y depuradores inteligentes de Microsoft (Pylance, debugpy).
Entornos interactivos de experimentación (Jupyter Notebooks).
Asistentes de formato y código estructurado (Ruff, isort, Python Indent).

Reporte de Cierre y Próximos Pasos:

El script finaliza ejecutando pruebas de importación reales. Realiza un test crítico importando tensorflow e inspeccionando el hardware para listar si existen tarjetas de video **GPU** dedicadas (vía CUDA) o si el entrenamiento correrá de manera estándar por **CPU**.





Pipeline de Instalación del Entorno de Trabajo

Fase 4: Chequeo y Diagnóstico del Entorno (check_env.py)

Esta fase representa la **auditoría y control de calidad** de todo el laboratorio

- **1. Auditoría del Interpretador Principal**

- **Función:** Identifica los datos duros de la instalación de Python.
- **Diagnóstico:** Reporta la versión exacta en ejecución y, fundamentalmente, la **ruta física del ejecutable (sys.executable)**. Esto es vital para confirmar que el sistema está usando el Python del laboratorio y no una versión vieja o secundaria instalada previamente en la máquina.

- **2. Test de Integración de Librerías Científicas**

El script no solo mira si los archivos existen; **intenta importarlos en tiempo real** en la memoria RAM para comprobar su integridad física:

- **ML e Inteligencia Artificial:** Valida el motor de **TensorFlow** y las librerías matemáticas de soporte como **NumPy, Pandas, SciPy y Scikit-Learn**.
- **Infraestructura de Gráficos y Datos:** Comprueba módulos visuales (**Matplotlib, Seaborn, Pillow**) y gestores de archivos de modelos complejos (**h5py**), asegurando que el pipeline de datos no sufra interrupciones por falta de código secundario.

- **3. Inspección Física de Hardware (Módulo Crítico)**

Esta sección interroga directamente al núcleo de TensorFlow para mapear la potencia de procesamiento de la computadora:

- **Mapeo de GPU:** Rastrea la presencia de placas de video dedicadas compatibles (NVIDIA CUDA). Si las encuentra, expone su nombre y modelo, confirmando que los entrenamientos pesados se delegarán a la tarjeta gráfica.
- **Modo CPU de Respaldo:** Si no detecta una GPU apta, emite una advertencia informativa notificando que el entorno está operativo, pero ejecutará los cálculos matemáticos utilizando el procesador central de la máquina de forma estándar.





Pipeline de Instalación del Entorno de Trabajo

Fase 4: Chequeo y Diagnóstico del Entorno (check_env.py)

- **4. Escaneo de Herramientas Externas en el PATH**
 - **Función:** Realiza un rastreo en las variables de entorno globales de Windows para verificar la disponibilidad de los programas satélite.
 - **Control:** Confirma el acceso inmediato por comandos a **Git** (control de cambios), **GitHub CLI** (enlace a la nube) y **Visual Studio Code** (editor de código), garantizando que el desarrollador pueda abrir su espacio de trabajo y subir código sin bloqueos de permisos.
- **Dictamen Final del Laboratorio**

El proceso concluye consolidando todos los datos recolectados en una pantalla de **Estado General**:
- **Entorno Verde (100% OK):** Si todas las pruebas de importación y herramientas responden de forma positiva, el sistema da luz verde para iniciar los scripts didácticos de simulación y el entrenamiento con MNIST.
- **Entorno en Alerta (FAIL):** Si algún componente falla, el reporte señala con precisión quirúrgica el módulo ausente o mal configurado para que el usuario pueda corregirlo inmediatamente sin tener que adivinar dónde está el error.





Mi primer programa en Python:

"Hello World" y verificación del entorno:

En la jerga de programación, el programa "Hola, Mundo" (o "Hello, World" en inglés) es un ejemplo sencillo y tradicional que muchos programadores escriben como su primer programa en un nuevo lenguaje de programación.

Propósito:

Verificación del entorno de desarrollo: Ayuda a verificar que el compilador o el intérprete del lenguaje está instalado y configurado correctamente.

Prueba de compiladores y depuradores: Asegura que el compilador puede convertir el código fuente en un programa ejecutable y que las herramientas de depuración (debuggers) están funcionando como se espera.

Prueba de Librerías instaladas: Asegura que el compilador puede convertir el código fuente en un programa ejecutable y que las herramientas de depuración (debuggers) están funcionando como se espera.

Pasos:

- 1- Iniciar el VSCode
- 2- File-New text file
- 3-Select Language – Python

```
print("Hello world")
```

```
print("-----")
print("Importando Libreria TF...")

import tensorflow as tf

print("Version tensorflow: ")
print(tf.__version__)
print("-----")
```

```
# Lista de librerías a verificar
librerias = ['numpy', 'pandas', 'matplotlib', 'tensorflow']
for nombre_libreria in librerias:
    try:
        libreria = __import__(nombre_libreria)
        print(nombre_libreria + " está instalado correctamente.
Versión: " + libreria.__version__)
    except ImportError:
        print(nombre_libreria + " no está instalado.")
```



```
> print("Gracias !")
> print("dudas, consultas...")
> input("Presione una tecla para salir...")
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep. Argentina 786, Sunchoales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383

